

Javanese Script Detection Using YOLO v8 Model

A¹, B², C³

^{1,2,3}Informatics Department, Faculty of Information Technology, Universitas Kristen Dut
Wacana Yogyakarta, Indonesia 55224

Abstract –

Keywords:

applied using ordinary computer equipment as well as mobile devices such as cellular phones. In simple terms, OCR is a technology that converts writing in digital images into text. Currently, its use is very broad, ranging from automated note processing in accounting, scanning of banking documents, health, commerce, traffic analysis and policing, to election data entry. OCR is also used for digitizing document content in the context of document content storage and becomes an alternative to document preservation in addition to storing it in the form of document images.

In addition, OCR is the initial stage for text processing in images in the field of Natural Language Processing (NLP), namely transcription (extracting text from images and rewriting it), transliteration (extracting text from one writing system and then rewriting it in another writing system), and translation (extracting text and then writing the translation). An example of the application of transliteration and translation is the Android application “Google Lens” which is able to extract the characters on the objects in the photo and then display the translated text according to the desired destination language.

In the context of preserving regional culture in Indonesia, OCR technology can be utilized in a way such as the “Google Lens” application to transliterate or transliterate writing in regional language script by displaying the reading text in Latin script used to write Indonesian. The use of OCR for this is considered to be the latest learning needs[1], because currently the use of local languages in Indonesia is decreasing, because the younger generation tends to use Indonesian as the main language of communication from an early age[2]. On the other hand, if local languages and scripts are not preserved, then Indonesia's cultural diversity

I. INTRODUCTION

Optical Character Recognition (OCR) is one of the information technologies that are widely used today. The sophistication of current hardware performance has allowed OCR to be

could become just a story in the future. Likewise, local wisdom passed down from generation to generation in local languages and manuscripts in local characters will be destroyed because no one can speak or read the language anymore.

In particular for quick use to scan writing in digital images, programming libraries and APIs (Application Programming Interface) for OCR of Latin characters have already existed, complete and near perfect, such as Tesseract created by Hewlett Packard and sponsored by Google[3]. Tesseract has also been equipped with grammar processing engines for a number of major Latin languages such as English, French, German and Indonesian. Tesseract has also been able to OCR several writing systems derived from the Brahmic Script such as Devanagari (India), Thai, Khmer and Laotian which are also used by a number of Southeast Asian nations[4]. Tesseract is also capable of extracting text in various image contexts such as book pages, packaging labels, signage and others. Tesseract versions 3.04 and above are said to support the Javanese script but have not been tested for accuracy, especially for variations in the Javanese script style.

The use of Optical Character Recognition (OCR) related to the digitization of Javanese characters in the Nusantara, especially for Javanese characters, has been carried out by a number of researchers. Recent examples include research that preprocessed images with crop and resize and utilized Convolutional Neural Network (CNN) for data training and testing processes, which resulted in an average accuracy of 70.63% for Javanese script with handwriting using ballpoint pen[5]. CNN was also used in another research which also used handwritten datasets. Preprocessing was not mentioned but from the example data included, it can be considered equivalent to crop and resize because the script size is the same. The total dataset used is 1050 images with 35 classes as training and test data. With 60 epochs and 6 cycles, this research resulted in a final accuracy of 84%[6]. Both of these studies used *nglegena* or *carakan* script classes only as their research objects.

Research on Nusantara writing systems similar to Javanese script (*abugida*), namely Sundanese script, has also been carried out for example with a preprocessing approach using

Canny edge detection and the main processor of Artificial Neural Network (JST) with back-propagation. This research focuses on 7 characters in the swara script group that have morphological properties such as *nglegena* script in Javanese script. The features used as JST input are metric and eccentricity, which resulted in test data accuracy of 76.1905%[7]. Another research applied CNN to 20 Lampung script characters with similar morphology to Javanese *nglegena* script. This study used 3000 data with 20 classes, and rescale preprocessing was carried out. The CNN used comes from the Python library “Tensorflow” and is executed using the Google Colab platform. With 10 epochs and a learning rate of 0.0001, it achieved 100% accuracy, precision and recall respectively. However, this excellent result is not supported by the presentation of the dataset form used in the published paper[8].

The aforementioned researches emphasized the dominance of neural networks as a current OCR tool. This research also followed the steps using YOLO v8 which also employs neural network.

II. RESEARCH METHODS

This research will employ YOLO v8 as the main model to detect basic Javanese script, which both are briefly explained below, and it will follow the research method further discussed.

A. YOLO v8

As a neural network model, YOLOv8 with its updated architecture and algorithm has revolutionized the field of object detection as it enables accurate and efficient object detection in real time. To achieve accurate and efficient object detection, Convolutional Neural Network (CNN) has emerged as the most effective approach. CNNs, which are inspired by biological visual systems, can learn to extract meaningful features from images and make predictions about the presence and location of objects. YOLOv8 leverages the power of CNN to achieve real-time object detection with high accuracy, as shown in Figure 1, where all input images are resized to 448×448, processed through a single CNN and

followed by non-maximum suppression of candidate bounding boxes[9].

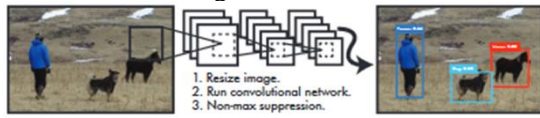


Figure 1 YOLO Detection System[9]

To determine the bounding box candidates, YOLO which was developed since v1 in 2015 uses a regression approach, which breaks the input image into an $S \times S$ grid which is further broken down into a set of bounding box candidates along with their confidence values and class probability maps based on the training data. Everything is then combined through tensor encoding to get the final bounding box detection[9].

The YOLO architecture has 24 convolution layers followed by 2 fully connected layers. The 1×1 convolutional layers alternate, reducing the feature space of the previous layer. YOLO pre-trains the convolutional layers on ImageNet classification at half resolution (224×224 input image) and then doubles the resolution for detection[9].

B. Javanese Script

Javanese script is a traditional writing used to represent the Javanese language and several other languages in Indonesia[5] such as Sundanese. Javanese script belongs to the abugida type of writing system, where each script represents a syllable with an inherent vowel and requires diacritics to change the vowel sound or eliminate it[10].

In Javanese script, the inherent vowel is the 'o' sound as in the English word “gone”, or transcribed using the IPA symbol 'o' and romanised as 'a'. The basic script that carries this inherent vowel in Javanese script is called *nglegena* or *carakan* script. There are 20 *nglegena* characters and their non-vowel forms (*pasangan*), and a number of diacritics (*sandhangan*), numerals, *Murda* characters and punctuation marks. Currently, Javanese script is supported on computer screens through Javanese UNICODE Block U+A980..U+A9DF (96 code points). The Javanese UNICODE Block table can be accessed from <https://www.unicode.org/charts/PDF/UA980.pdf>. The *nglegena* or *carakan* script is shown in Figure 2.



Figure 2 Javanese *carakan* script, *nglegena*[5]

There are a number of writing styles (*langgam*) of Javanese script that fall into three broad categories: *mbata sarimbag* (boxed), *ngetumbar* (rounded), and *mucuk eri* (has sharp edges) or a combination of the three[11]. The font style shown in Figure 2 is Surakarta-style *ngetumbar*, while the Provincial Government of Yogyakarta Special Region has stipulated that official documents and signage in the province use the *ngetumbar* font style called 'NYK Ngayogyan' which is the result of a font study by the Javanese Script Congress Team of the Language and Literature Section of the Yogyakarta Special Region Culture Office in 2020 with developer Apri Nugroho. Figure 3 shows the *carakan* script written in the NYK Ngayogyan font.

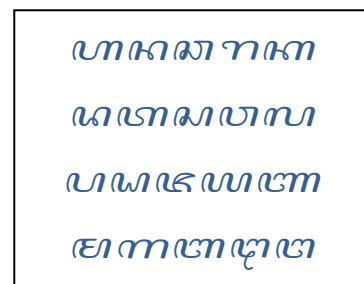


Figure 3 Javanese *carakan* script, *nglegena* in NYK Ngayogyan font

C. Research Method

This research is conducted using a single method described in Figure 4, which started with building a YOLO v8 model with n images, divided into 80:20 percentage of training and validation data. Although following a single workflow, we conducted two different experiments with different focus: (1) experiment focused on how singular characters

from Javanese script written on a readable surface will be detected, and (2) how characters on the same script, when occurred on a scene of mixed object as if we encounter them on public places as name plates or directions, will be detected.

After the training data is prepared, we train it using the default YOLO v8 model (yolov8n.pt) using Ultralytics Python library. Ultralytics is a specialised library designed for operations regarding the YOLO model. This step resulting in a new custom model, we named “best.pt”

This custom model then tested in Python against 60 test data: 30 generated using the NYK Ngayogyan font, and 30 using other methods.

After that the same model is converted into TFLite format to use in Android, and after conversion it is integrated into the Android app. We perform the second test using the Android app to get the direct image from the phone camera, and compare the results to the one obtained in Python. This is to test the real-time performance of the YOLO v8 model.

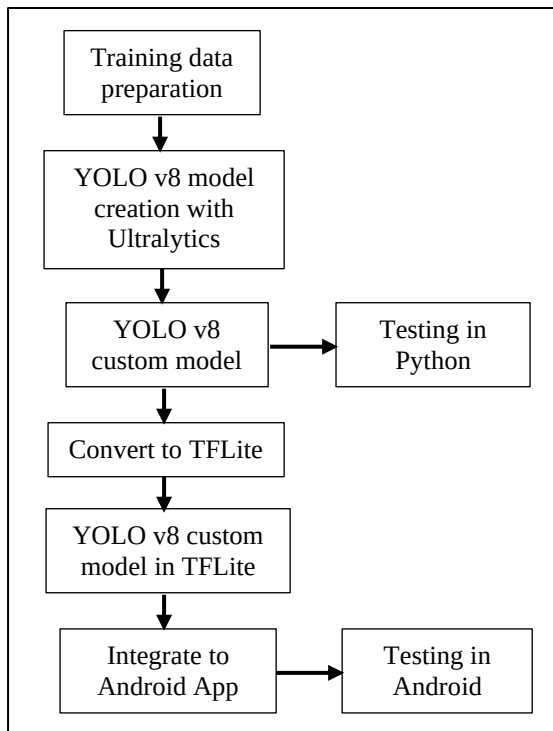


Figure 4 Javanese *carakan* script, *nglegena* in NYK Ngayogyan font

For our initial research, these two focuses are performed differently due to time and resource limitations, also we focus to detect only the basic 20 characters of the Javanese script called *carakan* or *nglegena*, and using the font

recently established as the cue font for Daerah Istimewa Yogyakarta, as described in point B. Therefore there are two models which are tested separately against different test data adjusted to each focus. The main difference of the two models are the training dataset used to build, which are explained below.

D. First Model

The first model, since it was focused to detect single characters written on a clear untangled surface like someone writing on a paper, a blackboard or writings on a clear concise plaque, the dataset consists of 200x200 images containing single javanese characters in white RGB(255,255,255), augmented with multiple colour backgrounds, different positions, variation in size (max. 150%), and slight variation (max. 30 degrees) in rotation.

The selected augmentation is assumed to represent how the geometric distortion that the characters endure when aimed using a camera and taken as photograph. The person taking the picture might not achieve an ideal condition to photograph a writings, as it may be a bit rotated, has a different standing point, and so on.

Example of data used for this first model is shown in Figure 5.

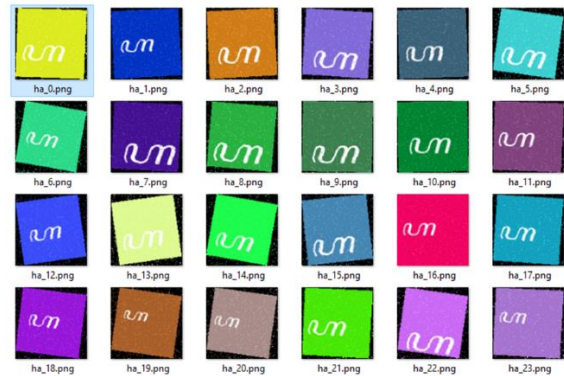


Figure 5 Model 1 dataset example for class “ha”

There are a 200 data for every class, split into 80 data for training and 20 data for validation.

E. Second Model

The second model, aimed to detect characters situated in a mixed scene, has a different augmentation. These images are bigger in size (640x640), do not have geometric variations but have a varied background and also various composition method. Also due to limited resource, we put all the 20 characters altogether into every background and use position maps to

indicate the location of each character as they belong to different classes.

This augmentation is selected to represent the condition occurred in real world when people photograph name plates, landmarks or directional sign containing Javanese characters. Example of data used for the second model is shown in Figure 6.



Figure 6 Model 2 dataset example for all classes

This model also used a hundred data for every class, split into 80 data for training and 20 data for validation.

III. RESULTS AND DISCUSSION

A. Results of First Model

The first model is tested twice as mentioned above, and each test is done against 60 images of two types: 30 images containing generated Javanese character using NYK Ngayogyan font (type A) and 30 images containing handwriting character (type B). Both only contains one Javanese character per image. The results are shown in Table 1 and Table 2, where the model worked satisfactorily. Table 1 is tested using Python, while Table 2 is tested from Android app. The Python testing is done by loading an image into the program written in Python, while the Android testing is done by aiming the camera at the same object displayed on screen and obtain the result.

Table 1 Results of first model testing (Python)

Test Data	No. of images	Correctly detected	Correctly classified	Percent age
Type A	30	30	30	100%
Type B	30	30	29	96,7%
Total	60	60	59	98,3%

Table 2 Results of first model testing (Android)

Test Data	No. of images	Correctly detected	Correctly classified	Percent age
Type A	30	30	30	100%
Type B	30	30	28	93,3%
Total	60	60	58	96,7%

Examples of the testing condition and results are shown in Figure 7 (Python test) and Figure 8 (Android test).



Figure 7 Example of first model result (Python)

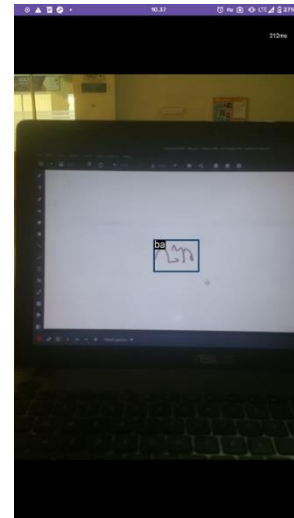


Figure 8 Example of first model result (Android)

We also conducted extra experiments involving images containing more than one Javanese character. The model happens to be capable of recognising more than one character on the image with several conditions: each character are clearly apart and for real-time conditions it should have no more than 5 character in a frame to avoid confusion for handwritten characters.

B. Results of Second Model

The second model is tested as the first model with a different test data as to fulfil the focus. We used 40 images of two types: 20 images

containing generated Javanese character using NYK Ngayogyan font (type A) and 20 images containing Javanese character in real life, for example from building name plaques, signs, etc. (type B). Both contains more than one Javanese character per image, and may mixed with Latin characters as Javanese signs usually bilingual with Indonesian. The results are shown in Table 3 and Table 4, where the model worked satisfactorily for type A but failed for type B.

Table 3 Results of second model testing (Python)

Test Data	No. of images	No. of chars.	Correctly detected chars.	Correctly classified chars.	Percentage
Type A	20	400	400	400	100%
Type B	20	148	43	20	13,5%
Total	40	548	443	420	

Table 4 Results of second model testing (Android)

Test Data	No. of images	No. of chars.	Correctly detected chars.	Correctly classified chars.	Percentage
Type A	20	400	400	386	96,5%
Type B	20	148	47	32	21,6%
Total	40	548	447	428	

The example of Type A second model testing is illustrated by Figure 9, where it was tested against one of the validation data, while the Type B testing is shown in Figure 10, when it was tried against a photo of a place name plaque.



Figure 9 Example of first model result (Type A - Python)

The result of Type B for test performed using Android is higher than when performed using Python because in real-time, the bounding box is moving several times from correct position into incorrect position and vice versa, so we record most correct position within 30 seconds of observation for each image. The similar is happened for classification, we record the most correct character classified within the given time. This may not reflect an exact result but show the possibilities of correct detection when the model is used in real-time.



Figure 10 Example of first model result (Type B - Android)

C. Discussion

It is interesting that while YOLO v8 is a deep learning model which ideally trained using at least 5000 data per class[12], with minimum amount of augmented training data, it can give a good detection performance against object with near-same shape, which in this research are Javanese character rendered using the same font as shown in the first model result test.

The first model result test also shown that models trained with geometry augmentation of printed characters from a single font can be potential in detecting handwritten characters which may not exactly have the same shape with the training characters.

On the other hand, when the data is not augmented with geometry variations, its performance reduced sharply due to narrowing detection, as it only detects character shapes which is almost identical to the training data, as in the case of the second model, although it does pass the hurdle of segmenting the characters from the rest of the image, even in a complex mixture of elements (shown by the 100% correctly detected characters in second model testing result, type A).

IV. CONCLUSION

YOLO v8 has the potential to use as model for OCR engine embedded in portable devices as shown from the discussed results. It has the potential of speed and portability, but the minimum amount of data trained in this research may have hinder its full potential.

If the two models can be combined and further augmented until the ideal amount of training data is achieved, it is possible to achieve greater results, and the final model could be rendered usable to achieve as a real-time Javanese transliterator that can be used to read Javanese script from those in the signs and those found in the book. Future researches could address this problem in two ways: (1) prepare the suitable training dataset and (2) to create a model suitable for the task.

V. ACKNOWLEDGEMENT

The authors would like to thank Lembaga Penelitian dan Pengabdian kepada Masyarakat (LPPM) Universitas Kristen Duta Wacana who have provided funding for this research.

REFERENCES

- [1] da Silva, I. A. L., Bernal-Merino, M. A., and Esqueda, M. D., "Transaction and Digital Technology: Practices, Theories, Research Methods, and The Classroom," *Belas Infiéis*, vol. 9, no. 4, pp. 3-13, 2020.
- [2] Cohn, A. C. and Ravindranath, M., "Local Languages in Indonesia: Language Maintenance or Language Shift?" *Jurnal Linguistik Indonesia*, vol. 32, no. 4, pp. 131-148, 2014.
- [3] Sharma, B. and Rao, A. K., "OCR Related Technology Methods," *International Journal of Advanced Trends in Computer Science and Engineering*, vol 9, no. 3, pp. 2789-2793, 2020.
- [4] Tesseract documentation, "Languages/Scripts supported in different versions of Tesseract". Last accessed Feb 2024. [Online]. Available: <https://tesseract-ocr.github.io/tessdoc/Data-Files-in-different-versions.html>
- [5] Pradhana, S. C. A., Wisesty, U. N. and Sthevanie F., "Pengenalan Aksara Jawa dengan Menggunakan Algoritma Convolutional Neural Network," *e-Proceeding of Engineering*, vol. 7, no. 1, pp. 2558-2567, 2020.
- [6] Jonathan, A. and Wasito, I., "Perancangan Aplikasi Pengenalan Aksara Jawa Digital Menggunakan Convolutional Neural Network Dan Computer Vision". *DECODE: Jurnal Pendidikan Teknologi Informasi*, vol. 3., no. 2, pp. 364-377, 2023.
- [7] Amalia, N., Hidayat, E. W., and Aldya, A. P., "Pengenalan Aksara Sunda Menggunakan Metode Jaringan Saraf Tiruan Backpropagation Dan Deteksi Tepi Canny," *CESS (Journal of Computer Engineering System and Science)*, vol. 5, no. 1, pp. 19-27, 2020.
- [8] Sazqiah, N. P., Mulyani, Y., Muhammad, M. A., Martinus, Sukmana, I., Nama, G. F., Huda, Z., Septiana, T., Kurniawan, P., and Djausal, G. P., "Pengenalan Aksara Lampung Menggunakan Metode CNN (Convolutional Neural Network)," *Prosiding Seminar Nasional Keinsinyuran (SNIP)*, 2022, pp. 1-5.
- [9] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A., "You Only Look Once: Unified, Real-Time Object Detection," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [10] Mahastama, A. W., "Model Berbasis Aturan Untuk Transliterasi Bahasa Jawa

- Dengan Aksara Latin Ke Aksara Jawa,”
Jurnal Buana Informatika (JBI), vol. 13,
no. 2, pp. 146–154, 2022.
- [11] Perdana, A. B., “Ragam Langgam Aksara
Jawa dari Manuskrip hingga Buku Cetak,”
Jurnal Manassa “Manuskripta”, vol. 10,
no. 1, pp. 1-28, 2020.
- [12] Goodfellow, I., Bengio, Y. and Courville,
A., “Deep Learning,” MIT Press, 2016.